1. Find the dataset's shape (# of rows, # of columns).

```
SELECT
	'# Records' AS Info
	, FORMAT(COUNT(*), 0) AS 'Value'
FROM sales s LEFT JOIN products p ON p.p_product_key = s.s_product_key
UNION ALL
SELECT
	'# Attributes'
	, (SELECT MAX(ORDINAL_POSITION) FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME = 'products')
		+ ((SELECT MAX(ORDINAL_POSITION) FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME = 'sales') );
```

2. Remove the leading & trailing spaces around the hyphen in values of column 'product_name'.

```
	SELECT REGEXP_REPLACE(product_name, "\\s*-\\s*", '-') AS product_name FROM products;
	-- REPLACE (as in query ahead) performs only literal, fixed-string substitutions. It cannot match variable whitespaces:
	SELECT REPLACE(REPLACE(product_name, ' -', '-'), '- ', '-') AS product_name FROM products;
```

3. Shift the 'price' column beside 'due_date' and then 'quantity' column beside 'price' column

```
	SELECT * FROM sales;                                          -- check column order
	ALTER TABLE sales MODIFY COLUMN price INT AFTER due_date;     -- shifting column
	SELECT * FROM sales;                                          -- check column order again
	ALTER TABLE sales MODIFY COLUMN quantity INT AFTER price;     -- shifting column
	SELECT * FROM sales;                                          -- check column order again (columns will be found re-ordered)
```

4. In 'products' table, fetch details of all red-colored products (product 'Taillights - Battery-Powered' shouldn't be in o/p though it has 'red' in it).

```
	-- COLLATE makes comparison case-sensitive, 'product_name' having 'Tailored' is not returned
	SELECT product_name FROM products WHERE product_name REGEXP "^.*Red.*" COLLATE utf8mb4_bin;
	SELECT product_name FROM products WHERE product_name LIKE "%Red%" COLLATE utf8mb4_bin;
```

5. Count the number of NULL/NA values in each column of table products.

```
SELECT 'p_product_key' AS 'Column_Header', SUM(CASE WHEN p_product_key IS NULL OR p_product_key = 'NA' THEN 1 ELSE 0 END) AS NULLS_Count FROM products UNION ALL
SELECT 'product_id', SUM(CASE WHEN product_id IS NULL OR product_id = 'NA' THEN 1 ELSE 0 END) FROM products UNION ALL
SELECT 'product_number', SUM(CASE WHEN product_number IS NULL OR product_number = 'NA' THEN 1 ELSE 0 END) FROM products UNION ALL
SELECT 'product_name', SUM(CASE WHEN product_name IS NULL OR product_name = 'NA' THEN 1 ELSE 0 END) FROM products UNION ALL
SELECT 'category_id', SUM(CASE WHEN category_id IS NULL OR category_id = 'NA' THEN 1 ELSE 0 END) FROM products UNION ALL
SELECT 'category', SUM(CASE WHEN category IS NULL OR category = 'NA' THEN 1 ELSE 0 END) FROM products UNION ALL
SELECT 'subcategory', SUM(CASE WHEN subcategory IS NULL OR subcategory = 'NA' THEN 1 ELSE 0 END) FROM products UNION ALL
SELECT 'maintenance', SUM(CASE WHEN maintenance IS NULL OR maintenance = 'NA' THEN 1 ELSE 0 END) FROM products UNION ALL
SELECT 'cost', SUM(CASE WHEN cost IS NULL OR cost = 'NA' THEN 1 ELSE 0 END) FROM products UNION ALL
SELECT 'product_line', SUM(CASE WHEN product_line IS NULL OR product_line = 'NA' THEN 1 ELSE 0 END) FROM products UNION ALL
SELECT 'start_date', SUM(CASE WHEN start_date IS NULL THEN 1 ELSE 0 END) FROM products
ORDER BY NULLS_Count DESC;
```

-- A query can be written which will **dynamically** generate the above statements (except the ORDER BY at the end) for copy-paste-run, as described ahead:

```
SET SESSION group_concat_max_len = 10000;     -- otherwise GROUP_CONCAT's result in output will be trimmed to first 1024 bytes
SELECT
       GROUP_CONCAT(
         CONCAT(
           "SELECT ", COLUMN_NAME, " AS Column_Header, ",
           "SUM(CASE WHEN ", COLUMN_NAME, " IS NULL OR ", COLUMN_NAME, " = 'NA' THEN 1 ELSE 0 END) AS NULLS_count ",
           "FROM products"
               ) SEPARATOR " UNION ALL\n"
        ) AS generated_SQL
FROM INFORMATION_SCHEMA.COLUMNS WHERE LOWER(TABLE_NAME) = 'products' AND TABLE_SCHEMA = 'your_db_name_here';
```

6. Fetch those dates of the calendar on which there was zero sales.

```
SELECT MIN(order_date), MAX(order_date) FROM sales INTO @min_dt, @max_dt;            -- 2010-12-29, 2014-01-28
SET @@cte_max_recursion_depth = 1500;
WITH RECURSIVE CTE_Dates AS (
    SELECT DATE(@min_dt) AS daily_date
    UNION ALL
    SELECT daily_date + INTERVAL 1 DAY FROM CTE_Dates
    WHERE daily_date < @max_dt
)
SELECT
       daily_date AS zero_sales_dates
FROM CTE_Dates d LEFT JOIN sales s ON d.daily_date = s.order_date
WHERE s.order_date IS NULL;
```

```
-- Date range as a temporary table (CTE expires with query run; temporary table remains till session end (or is dropped) so can be queried by many SELECTs)
       CREATE TEMPORARY TABLE date_range AS
       WITH RECURSIVE CTE_Dates AS (
            SELECT DATE(@min_dt) AS daily_date
            UNION ALL
            SELECT daily_date + INTERVAL 1 DAY FROM CTE_Dates
            WHERE daily_date < @max_dt
        ) SELECT * FROM CTE_Dates;
        DROP TEMPORARY TABLE IF EXISTS date_range; -- Keyword TEMPORARY is optional, makes intent clear though
```

7. Which category hasn't been classified into any of the product lines?

```
SELECT DISTINCT category FROM products WHERE product_line = 'NA' ;
```

8. Find the range of sales for each sub-category during 2013.

```
SELECT
       subcategory
       , FORMAT(MIN(sales_amount), 2) AS min_sales_amt
       , FORMAT(MAX(sales_amount), 2) AS max_sales_amt
FROM sales s LEFT JOIN products p ON p.p_product_key = s.s_product_key
WHERE YEAR(order_date) = 2013
GROUP BY subcategory ;
```

9. Rolling average sales of each category (window: 2 previous months and current month).

```
WITH CTE_Monthly_Sales AS (
SELECT
	DISTINCT category, MONTH(order_date) AS 'Month', ROUND(SUM(sales_amount), 2) AS sales
FROM products p LEFT JOIN sales s ON p.p_product_key = s.s_product_key
WHERE category IS NOT NULL AND order_date IS NOT NULL
GROUP BY category, MONTH(order_date)
ORDER BY category, 'Month'
)
SELECT *
	, ROUND(AVG(sales) OVER(PARTITION BY category ORDER BY 'Month' ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS prev_3_months_avg_sales
FROM CTE_Monthly_Sales ;
```

10. Find the count of mountain bikes and road bikes ordered from Apr 2012 till Mar 2013.

```
SELECT
	subcategory AS Subcategory
	, COUNT(*) AS Units
FROM sales s LEFT JOIN products p ON p.p_product_key = s.s_product_key
WHERE subcategory IN ('Road Bikes', 'Mountain Bikes') AND order_date BETWEEN '2012-04-01' AND '2013-03-31'
GROUP BY subcategory ;
```

11. Find the cumulative sales of products over all the months of 2011 and 2012.

```
WITH CTE_Monthly_Sales AS (
SELECT
	YEAR(order_date) AS order_year
	, product_number
	, product_name
	, MONTH(order_date) AS 'Month'
	, SUM(sales_amount) AS sales
FROM sales s LEFT JOIN products p ON p.p_product_key = s.s_product_key
WHERE order_date IS NOT NULL AND YEAR(order_date) IN (2011, 2012)
GROUP BY YEAR(order_date), product_number, product_name, MONTH(order_date)
ORDER BY sales DESC
)
SELECT *
	, SUM(sales) OVER(PARTITION BY order_year, product_name ORDER BY order_year, Month) AS cumulative_sales
FROM CTE_Monthly_Sales ;
```

12. Which red coloured products were never sold?

```
SELECT
	product_id, product_number, product_name, order_date, sales_amount
FROM products p LEFT JOIN sales s ON p.p_product_key = s.s_product_key
WHERE product_name LIKE '% Red%' AND sales_amount IS NULL
ORDER BY product_id ;
```

13. List the subcategories of each category.

```
SELECT
	DISTINCT category AS Categories
	, GROUP_CONCAT(DISTINCT subcategory SEPARATOR ', ') AS Subcategories
FROM products WHERE category IS NOT NULL
GROUP BY category ;
```

14. Find the percentage contribution of each category's sales to overall sales.

```
WITH CTE_Category_Sales AS (
SELECT
    category
    , COALESCE(ROUND(SUM(sales_amount), 2), 0.00) AS sales
FROM products p LEFT JOIN sales s ON p.p_product_key = s.s_product_key
WHERE category IS NOT NULL
GROUP BY category
)
SELECT
    category
    , FORMAT(sales, 2) AS sales
    , FORMAT(SUM(sales) OVER(), 2) AS overall_sales
    , CONCAT(ROUND(sales/SUM(sales) OVER() * 100, 1), '%') AS '%_of_overall_sales'
FROM CTE_Category_Sales ;
```

15. Average price for each product line in 2012?

```
WITH CTE_avg_price AS (
	SELECT
		product_line
		, price
	FROM sales s LEFT JOIN products p ON p.p_product_key = s.s_product_key
	WHERE YEAR(order_date) = 2012
)
SELECT
	product_line
	, ROUND(AVG(price), 2) AS line_2012_avg_price
FROM CTE_avg_price
GROUP BY product_line ;
```

16. Find the lowest revenue generating subcategory of each category in the last quarter of 2013.

```
WITH CTE_lowest_revenue AS (
SELECT
	p.category
	, p.subcategory
	, SUM(sales_amount) AS total_sales
FROM sales s LEFT JOIN products p ON p.p_product_key = s.s_product_key
WHERE QUARTER(order_date) = 4
GROUP BY p.category, p.subcategory
ORDER BY category
)
```

```
SELECT
       category
       , subcategory
FROM (
              SELECT *
                     , DENSE_RANK() OVER(PARTITION BY category ORDER BY total_sales ASC) AS ranked
              FROM CTE_lowest_revenue
      ) temp
WHERE ranked = 1 ;
```

17. What were the net sales on weekends during the years 2011, 2012 and 2013?

```
SELECT
       order_year
       , FORMAT(SUM(total_sales), 2) AS net_weekend_sales
FROM (
       SELECT
              YEAR(s.order_date) AS order_year
              , WEEKDAY(s.order_date) AS order_weekday   -- Monday: 0, Sunday: 6
              , DAYNAME(s.order_date) AS order_dayname
              , SUM(s.sales_amount) AS total_sales
       FROM sales s LEFT JOIN products p ON p.p_product_key = s.s_product_key
       WHERE YEAR(s.order_date) IN (2011, 2012, 2013) AND DAYNAME(s.order_date) IN ('Saturday', 'Sunday')
       GROUP BY YEAR(s.order_date), WEEKDAY(s.order_date), DAYNAME(s.order_date)
       ) temp
GROUP BY order_year ;
```

18. Find each sub-category's annual sales, prev year sales, sales difference from prev year, sales change factor.

```
WITH CTE_Annual_Sales AS (
SELECT
       DISTINCT YEAR(order_date) AS order_year
        , subcategory
        , SUM(sales_amount) OVER(PARTITION BY subcategory ORDER BY YEAR(order_date)) AS sales_annual
FROM sales s LEFT JOIN products p ON p.p_product_key = s.s_product_key
WHERE subcategory LIKE '%Bikes%' AND order_date IS NOT NULL
)
SELECT
    order_year AS 'year'
    , subcategory
    , FORMAT(sales_annual, 2) AS sales_annual
    , FORMAT(LAG(sales_annual) OVER(PARTITION BY subcategory ORDER BY order_year), 2) AS sales_prev_year
    , FORMAT(sales_annual - LAG(sales_annual) OVER(PARTITION BY subcategory ORDER BY order_year), 2) AS sales_diff
    , ROUND((sales_annual - LAG(sales_annual) OVER(PARTITION BY subcategory ORDER BY order_year))/LAG(sales_annual) OVER(PARTITION BY subcategory
              ORDER BY order_year), 1) AS sales_growth_factor
FROM CTE_Annual_Sales ;
```

19. How many categories do not belong to any product line?

```
SELECT * FROM products where product_line = 'NA';      -- 17 records
SELECT
       COUNT(DISTINCT category) AS category_sans_product_line
FROM products WHERE product_line = 'NA' ;              -- 1  (only 'Components' category)
```

20. Fetch the details of the products that remained unsold over the years.

```
SELECT * FROM products WHERE p_product_key NOT IN (SELECT s_product_key FROM sales) ;    -- 165 rows
```

21. Which subcategories that went unsold between January 01, 2011 and December 31, 2013?

```
SELECT
       COUNT(DISTINCT subcategory) AS '# Unsold'
       , GROUP_CONCAT(DISTINCT subcategory SEPARATOR ', ') AS 'Unsold SubCategories'
FROM products
WHERE subcategory NOT IN (
       SELECT
              DISTINCT subcategory
       FROM sales s LEFT JOIN products p ON p.p_product_key = s.s_product_key
       WHERE order_date BETWEEN '2011-01-01' AND '2013-12-31'
) ;
```

22. Find the sales of S, M, L or XL-sleeved clothing over the years.

```
WITH CTE_Sleeved_Sales AS (
       SELECT
              product_name
              , sales_amount
              , order_date
       FROM products p LEFT JOIN sales s ON p.p_product_key = s.s_product_key
       WHERE product_name LIKE '%Sleeve%'
)
SELECT
    DISTINCT product_name AS Product
    , FORMAT(SUM(CASE WHEN YEAR(order_date) = 2012 THEN sales_amount ELSE 0 END), 2) AS '2012_Sales'
    , FORMAT(SUM(CASE WHEN YEAR(order_date) = 2013 THEN sales_amount ELSE 0 END), 2) AS '2013_Sales'
    , FORMAT(SUM(CASE WHEN YEAR(order_date) = 2014 THEN sales_amount ELSE 0 END), 2) AS '2014_Sales'
FROM CTE_Sleeved_Sales
GROUP BY product_name
ORDER BY Product ;
```

23. Quantities of subcategories ordered in each month during 2013

```
SELECT
    subcategory AS 'Sub-Cat_Ordered'
    , SUM(CASE WHEN MONTHNAME(order_date) = 'January' THEN 1 ELSE 0 END) AS 'Jan #'
    , SUM(CASE WHEN MONTHNAME(order_date) = 'February' THEN 1 ELSE 0 END) AS 'Feb #'
    , SUM(CASE WHEN MONTHNAME(order_date) = 'March' THEN 1 ELSE 0 END) AS 'Mar #'
    , SUM(CASE WHEN MONTHNAME(order_date) = 'April' THEN 1 ELSE 0 END) AS 'Apr #'
    , SUM(CASE WHEN MONTHNAME(order_date) = 'May' THEN 1 ELSE 0 END) AS 'May #'
```

```
    , SUM(CASE WHEN MONTHNAME(order_date) = 'June' THEN 1 ELSE 0 END) AS 'Jun #'
    , SUM(CASE WHEN MONTHNAME(order_date) = 'July' THEN 1 ELSE 0 END) AS 'Jul #'
    , SUM(CASE WHEN MONTHNAME(order_date) = 'August' THEN 1 ELSE 0 END) AS 'Aug #'
    , SUM(CASE WHEN MONTHNAME(order_date) = 'September' THEN 1 ELSE 0 END) AS 'Sep #'
    , SUM(CASE WHEN MONTHNAME(order_date) = 'October' THEN 1 ELSE 0 END) AS 'Oct #'
    , SUM(CASE WHEN MONTHNAME(order_date) = 'November' THEN 1 ELSE 0 END) AS 'Nov #'
    , SUM(CASE WHEN MONTHNAME(order_date) = 'December' THEN 1 ELSE 0 END) AS 'Dec #'
FROM sales s
LEFT JOIN products p ON p.p_product_key = s.s_product_key
WHERE YEAR(order_date) = 2013
GROUP BY subcategory
ORDER BY subcategory ;
```

24. Find the top-5 product names by average sales in each category for the year 2013 (if more than one product name in top-5 have same avg sales, include all of them)

```
WITH CTE_Avg_Sales AS (
       SELECT
              category
              , product_name
              , ROUND(AVG(sales_amount), 2) AS avg_sales
       FROM products p LEFT JOIN sales s ON p.p_product_key = s.s_product_key
       WHERE YEAR(order_date) = 2013
       GROUP BY 1, 2
       ORDER BY category, avg_sales DESC
),
CTE_Ranking AS (
       SELECT *
              , DENSE_RANK() OVER(PARTITION BY category ORDER BY avg_sales DESC) AS avg_sales_rank
       FROM CTE_Avg_Sales
)
SELECT
    category
    , GROUP_CONCAT(DISTINCT product_name SEPARATOR ', ') AS same_rank_products
    , avg_sales_rank AS 'rank'
FROM CTE_Ranking
WHERE avg_sales_rank BETWEEN 1 AND 5
GROUP BY category, avg_sales_rank ;
```

25. Create the report query for business metrics (as shown in the problemset).

```
SELECT 'Total Revenue' AS Item, FORMAT(ROUND(SUM(sales_amount), 2), 2) AS Value FROM sales
UNION ALL SELECT 'Average Price', ROUND(AVG(price), 2) FROM sales
UNION ALL SELECT 'Total Units Sold', SUM(quantity) FROM sales
UNION ALL SELECT 'Max Sales Order ID', order_number
FROM (SELECT order_number FROM sales
      WHERE sales_amount = (SELECT MAX(sales_amount) FROM sales)
      ORDER BY order_number DESC LIMIT 1
     ) temp
UNION ALL
SELECT 'Min Sales Order ID', order_number
```

```
FROM (SELECT order_number FROM sales
      WHERE sales_amount = (SELECT MIN(sales_amount) FROM sales)
      ORDER BY order_number DESC LIMIT 1
       ) temp
UNION ALL
SELECT 'Highest Sales Date', order_date
FROM (SELECT order_date FROM sales
      WHERE sales_amount = (SELECT MAX(sales_amount) FROM sales)
      ORDER BY order_date DESC LIMIT 1) temp
UNION ALL
SELECT 'Lowest Sales Date', order_date
FROM (SELECT order_date FROM sales
      WHERE sales_amount = (SELECT MIN(sales_amount) FROM sales)
      ORDER BY order_date DESC LIMIT 1) temp
UNION ALL
SELECT
       DISTINCT 'Highest Sales Category'
       , GROUP_CONCAT(DISTINCT p.category SEPARATOR '\r, ')
FROM sales s LEFT JOIN products p ON p.p_product_key = s.s_product_key
WHERE sales_amount = (SELECT MAX(sales_amount) FROM sales)
UNION ALL
SELECT
       DISTINCT 'Lowest Sales Category'
       , GROUP_CONCAT(DISTINCT p.subcategory SEPARATOR '\r, ')
FROM sales s LEFT JOIN products p ON p.p_product_key = s.s_product_key
WHERE sales_amount = (SELECT MIN(sales_amount) FROM sales) ;
```